

Arquitectura del sistema de subastas

Oct 5, 2026 · @Catalina Rodríguez

Proponemos una arquitectura SOFEA sobre microservicios, con un gateway de capa 7, dos microservicios y dos bases de datos (PostgreSQL y MongoDB), todo desplegado en contenedores. Este documento explica cada componente, cada conector y, sobre todo, el porqué de cada decisión, para que el equipo la revise antes de implementar.

Contexto

El proyecto es una plataforma web para una casa de subastas. Según la descripción del negocio, maneja tres modalidades de venta (subasta en evento, subasta sin evento y venta directa) que comparten usuarios, inventario, dinero y proceso de entrega.

Los requisitos no funcionales del enunciado son:

- Arquitectura distribuida.
- Al menos un componente de presentación (front-end web).
- Al menos dos componentes de lógica.
- Al menos dos componentes de datos: una base relacional y una NoSQL.
- Al menos dos tipos distintos de conectores basados en HTTP.
- Al menos tres lenguajes de propósito general.
- Despliegue orientado a contenedores.

La descripción del negocio agrega exigencias que condicionan la arquitectura: nadie puede pujar más de lo que tiene disponible, una pieza no puede venderse dos veces, las subastas deben cerrarse solas aunque no haya nadie conectado, y el historial de pujas y dinero nunca puede alterarse. Ante una situación inesperada, el sistema debe preferir detener una operación antes que inventar un estado incorrecto.

Estilo arquitectónico: SOFEA sobre microservicios

SOFEA (Service-Oriented Front-End Architecture) separa tres responsabilidades que en una aplicación web tradicional hace el servidor:

1. **Descarga de la aplicación:** el servidor entrega el código del front una sola vez.
2. **Flujo de presentación:** pantallas, navegación, estado de la interfaz y renderizado viven por completo en el navegador.
3. **Intercambio de datos:** el backend solo envía y recibe datos (JSON). Nunca genera HTML.

En una aplicación clásica (PHP, JSP) el servidor hace las tres cosas: arma cada página y decide la navegación. En SOFEA el servidor solo descarga la app al inicio e intercambia datos de ahí en adelante.

Detrás del front, la lógica se reparte en microservicios: procesos independientes, cada uno con su propia base de datos, que se comunican por red.

Por qué este estilo:

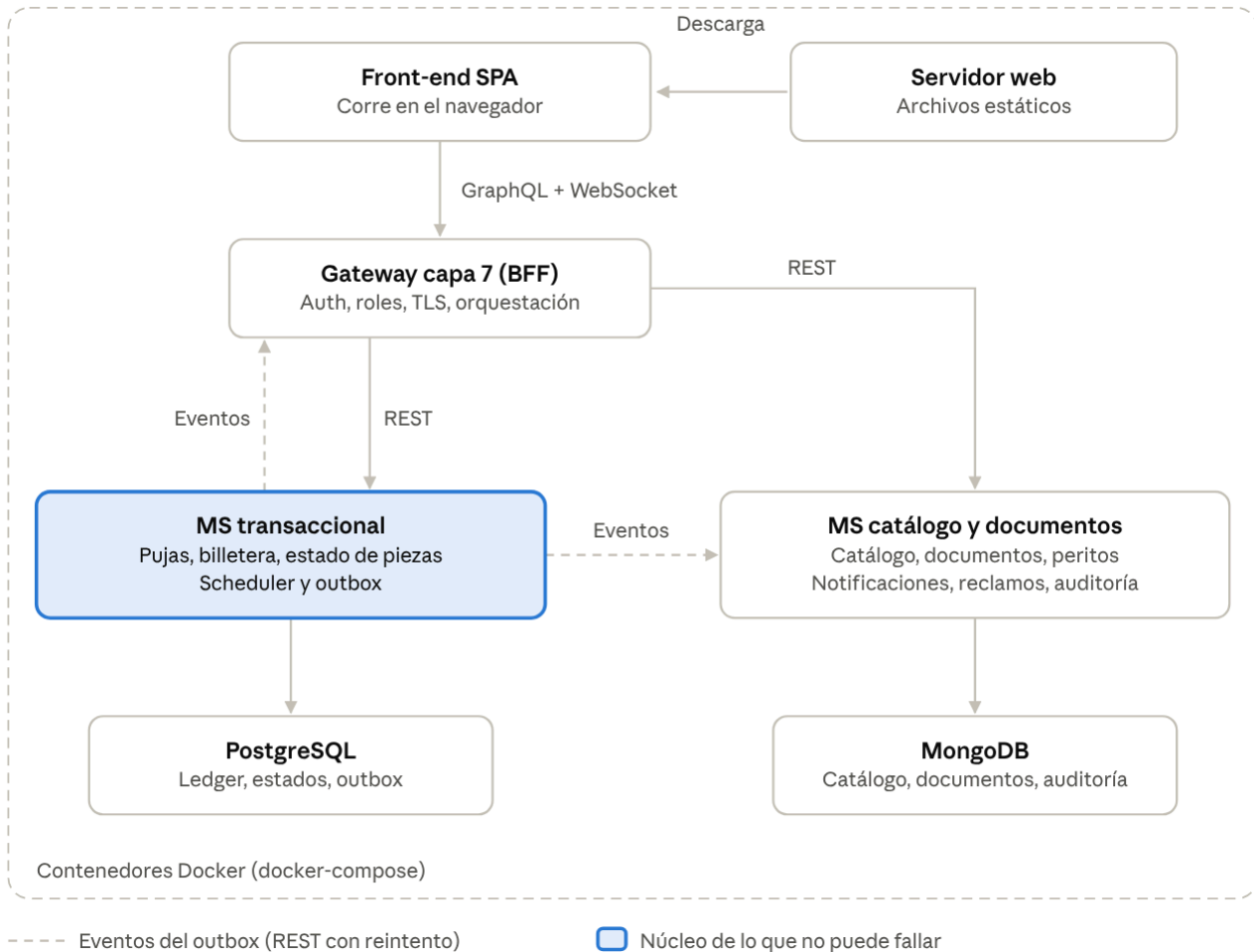
- **Desacople:** el front puede cambiar sin tocar los servicios, y los servicios pueden evolucionar o escalar sin romper el front.
- **Experiencia en tiempo real:** en una subasta el precio cambia constantemente. Una app que no recarga páginas y solo pide datos es la forma natural de mostrarlo.
- **Encaja con el enunciado:** da componentes de presentación, lógica y datos claramente separados, cada uno en su contenedor, lo que hace la arquitectura distribuida de verdad.

Restricción que se deriva: el front debe renderizarse en el cliente (React, Vue o Angular con Vite). No usamos SSR (Next.js o Nuxt en modo servidor), porque devuelve la presentación al servidor y rompe el principio de SOFEA.

Vista general

El sistema tiene seis componentes en contenedores: el navegador descarga la SPA del servidor web y, desde ahí, solo habla con el gateway.

El front solo habla con el gateway; lo crítico vive en el MS transaccional



Arquitectura del sistema de subastas · 6 componentes

El gateway llama por REST a los dos microservicios, y cada uno es dueño de su base. Las flechas punteadas son los eventos que el MS transaccional genera por su cuenta (cierres automáticos, "te superaron") y entrega al gateway, para empujarlos por WebSocket, y al MS catálogo, para crear la notificación.

Componentes

El sistema tiene seis componentes: un front-end, un servidor web, un gateway, dos microservicios y dos bases de datos. Para cada uno se explica qué hace y por qué existe.

Front-end SPA (presentación)

Qué hace: es la aplicación que corre en el navegador. Muestra el catálogo, el detalle de cada subasta, el formulario de puja, la billetera y el estado de las compras. Maneja la navegación y el estado de la interfaz, y actualiza el precio en vivo.

Por qué: una vez descargada, nunca vuelve a pedir HTML; solo pide datos. Eso da una experiencia fluida, sin recargas, que es indispensable cuando el usuario está mirando un precio que cambia.

Servidor web de archivos estáticos

Qué hace: entrega el HTML, JavaScript y CSS de la SPA cuando el usuario entra al sitio. No ejecuta lógica ni consulta bases de datos.

Por qué separado: es la fase de descarga de SOFEA. El front se despliega de forma independiente: si cambiamos el diseño, solo se redespiega este contenedor. Al servir solo archivos, es liviano y fácil de cachear. Si lo hiciera el gateway, mezclaríamos descarga con intercambio de datos, que es lo que SOFEA busca separar.

Gateway de capa 7 / BFF (lógica)

Qué hace: es la única puerta de entrada del front al backend. Trabaja en capa 7 (aplicación): decide según la ruta HTTP, los headers y el token, no solo reenvía tráfico. Sus responsabilidades:

- **Autenticación y roles:** valida el JWT e identifica el rol (visitante, comprador, vendedor, administrador, perito, logística). Rechaza de entrada lo que el rol no permite.
- **Rutas públicas y privadas:** un visitante puede ver catálogo, precio actual y tiempo restante sin cuenta. Pujar, comprar o ver la billetera exige sesión.
- **Terminación TLS:** los datos viajan cifrados entre el navegador y el sistema.
- **Rate limiting:** limita pujas e intentos de inicio de sesión para frenar abuso y fuerza bruta.
- **Anonimización del canal público:** por WebSocket, los observadores reciben "Postor #12 ofreció \$5.000", nunca la identidad real.
- **Acceso a subastas cerradas:** antes de abrir el WebSocket de un evento cerrado, verifica que el usuario esté inscrito y habilitado.
- **Orquestación:** recibe una consulta GraphQL, llama a los microservicios necesarios por REST, combina las respuestas y devuelve exactamente lo que la pantalla necesita.
- **Tiempo real:** mantiene las conexiones WebSocket y empuja a los clientes los eventos que le llegan del backend.

Por qué existe: esconde la estructura interna (el front no sabe cuántos servicios hay ni dónde están), evita que el navegador haga muchas llamadas y centraliza lo transversal (seguridad, permisos, límites) para no repetirlo en cada servicio. Es lo que hace a la arquitectura orientada a front: su API se diseña pensando en las pantallas, no en las tablas.

Lo que no hace: no tiene reglas de negocio. No decide si una puja es válida; eso lo decide el MS transaccional. Los microservicios vuelven a validar la regla fina (por ejemplo, "no puedes pujar por tu propia pieza") aunque el gateway ya haya revisado el rol. Si el gateway acumula lógica, se convierte en un monolito disfrazado.

MS transaccional (lógica)

Qué hace: es el núcleo de todo lo que no puede fallar:

- **Usuarios y verificación:** cuenta, roles y estado (activa, suspendida, limitada). Suspende cambia el estado; nunca borra compras, pujas ni pagos.
- **Ciclo de vida de la pieza:** la máquina de estados (registro, pendiente de revisión, pendiente de inspección, aprobada, disponible, comprometida, vendida, en entrega, retirada) y su ubicación de custodia.
- **Billetera:** dinero total, disponible y comprometido de cada usuario, llevado como libro contable.
- **Las tres modalidades de venta:** subasta en evento, subasta sin evento y venta directa con reserva temporal.
- **Proceso posterior a la venta:** confirmación, pago, transferencia de propiedad y entrega, como estados separados ("pagado" no es lo mismo que "propiedad transferida").
- **Scheduler:** cierra subastas, vence reservas y vigila plazos de pago según la hora guardada en la base.
- **Outbox:** publica los eventos que genera (cierre de subasta, "te superaron") hacia los demás componentes.

Por qué un servicio aparte: es la parte crítica, donde un error significa un ganador equivocado o dinero perdido. Aislarla permite protegerla, probarla y escalarla de forma independiente; la carga de búsquedas en el catálogo no le quita capacidad de recibir pujas.

MS catálogo y documentos (lógica)

Qué hace: maneja lo flexible y lo de alto volumen:

- **Catálogo y publicaciones:** fichas con atributos distintos por categoría, fotos, valor estimado y condiciones. Es la vista pública, sin datos privados del dueño.
- **Documentos de la pieza:** tasación, procedencia y anexos, con historial de versiones. Se descargan a través del servicio, nunca por URL directa.
- **Inspecciones y visitas de peritos:** solicitudes, propuestas de fecha y resultado. La decisión final (aprobada o rechazada) se comunica al MS transaccional, que es el dueño del estado de la pieza.

- **Notificaciones:** bandeja de cada usuario y alertas para administradores.
- **Reclamos:** cada uno con su propio historial, sin modificar la venta original.
- **Bitácora de auditoría:** quién hizo qué, cuándo y por qué en las acciones administrativas.

Por qué un servicio aparte: tiene un perfil distinto al transaccional: se lee mucho, se escribe sin transacciones complejas y los datos son heterogéneos. Separarlo permite usar la base que mejor se ajusta a ese perfil y que cada servicio falle sin arrastrar al otro.

Bases de datos

PostgreSQL pertenece al MS transaccional y MongoDB al MS catálogo. Se explican en detalle en la sección de bases de datos.

Decisiones clave de la división

La decisión más importante de la arquitectura es que la billetera vive en el mismo servicio y la misma base que las pujas.

La billetera va con las pujas

La descripción del negocio exige tres cosas sobre el dinero:

- Nadie puede ofrecer más de lo que tiene libre en su cuenta.
- Al pujar, el dinero queda apartado.
- Si alguien te supera, tu dinero se libera al instante, sin que lo pidas.

Si la billetera estuviera en un servicio y las pujas en otro, aceptar una puja serían dos operaciones en dos bases distintas: apartar el dinero en una y registrar la puja en la otra. Si la segunda falla después de la primera, queda dinero apartado sin puja; si ocurre al revés, queda una puja sin respaldo. Coordinar eso exige transacciones distribuidas o compensaciones, que son complejas y justo el tipo de falla que el negocio dice que no puede ocurrir.

Al tenerlas juntas, aceptar una puja es **una sola transacción de PostgreSQL**:

1. Bloquear la subasta y leer la oferta actual.
2. Validar que la nueva oferta supere la actual más el incremento mínimo.
3. Validar que el postor tenga fondos disponibles suficientes.
4. Apartar el dinero del nuevo postor.
5. Liberar el dinero del postor anterior.
6. Registrar la puja y, si corresponde, extender el cierre.
7. Guardar el evento "te superaron" en el outbox.

O pasa todo o no pasa nada.

El estado de la pieza vive en PostgreSQL, aunque la ficha esté en MongoDB

La ficha de la pieza (descripción, fotos, atributos) es información para mostrar y vive en el catálogo. El estado (disponible, comprometida, vendida) es una regla de negocio crítica: "no venderla dos veces". Ese estado tiene que poder bloquearse dentro de la misma transacción que la puja o la reserva, así que vive en el MS transaccional.

Prueba de que la división es correcta

Cada servicio debe poder caerse sin que el otro deje de funcionar. Si se cae el catálogo, las pujas en curso siguen funcionando. Si se cae el transaccional, la gente aún puede navegar los artículos y leer sus notificaciones.

Si el sistema creciera

El MS transaccional concentra mucho y, en un sistema de producción, lo natural sería dividirlo (por ejemplo, separar logística o usuarios). Billetera, pujas y estado de la pieza deberían seguir juntos por la razón anterior.

Bases de datos

Usamos PostgreSQL para lo que exige consistencia fuerte y MongoDB para lo que exige flexibilidad. Cada base pertenece a un solo servicio.

	PostgreSQL	MongoDB
Dueño	MS transaccional	MS catálogo y documentos
Modelo	Tablas relacionales	Documentos en colecciones
Guarda	Usuarios, piezas y su estado, subastas, pujas, billetera (ledger), ventas, outbox	Fichas del catálogo, documentos y versiones, inspecciones, notificaciones, reclamos, auditoría
Fortaleza que usamos	Transacciones ACID, bloqueos, llaves foráneas	Esquema flexible, datos embebidos, escala horizontal

PostgreSQL (relacional)

Por qué:

- **Transacciones ACID:** una puja se registra completa o no se registra.

- **Bloqueos y aislamiento:** resuelven las pujas simultáneas sobre la misma subasta.
- **Integridad referencial:** una puja siempre apunta a una subasta y un usuario que existen.
- **Restricciones en la base:** por ejemplo, un índice único parcial que impide más de una venta activa por pieza. La regla se cumple aunque el código tenga un error.
- **Estructura estable:** una puja siempre tiene usuario, monto y fecha. No hay variabilidad que justifique un esquema flexible.

La billetera como libro contable (ledger). Cada movimiento de dinero (consignación, apartado, liberación, pago, devolución, retiro, comisión, impuesto, cargo) es una fila que solo se inserta, nunca se actualiza ni se borra. El saldo disponible y el comprometido se calculan a partir de los movimientos. Un error se corrige con un movimiento de reversión, no editando el original. Esto cumple dos exigencias del negocio: que ningún administrador pueda borrar un movimiento financiero y que se pueda reconstruir cuánto dinero tenía un usuario en cualquier momento y por qué. La misma regla de solo inserción aplica a las pujas.

MongoDB (NoSQL de documentos)

Guarda documentos con formato parecido a JSON, agrupados en colecciones. Cada documento puede tener una estructura distinta y llevar objetos y listas anidados. Por ejemplo, dos fichas del catálogo:

```
{ "_id": "a101", "subastaId": 45, "titulo": "Mazda 3 2018", "categoria":
"vehiculos",
  "atributos": { "kilometraje": 62000, "transmision": "automatica" },
  "fotos": ["mazda1.jpg", "mazda2.jpg"] }
```

```
{ "_id": "a102", "subastaId": 46, "titulo": "Óleo paisaje andino",
"categoria": "arte",
  "atributos": { "autor": "J. Pérez", "tecnica": "óleo sobre lienzo" },
  "fotos": ["oleo1.jpg"] }
```

Por qué:

- **Esquema flexible:** la casa vende arte, joyas, antigüedades, coleccionables y vehículos. Cada categoría tiene atributos distintos. En una base relacional habría que crear una tabla por categoría o llenar columnas de nulos; en Mongo cada ficha trae lo suyo, y una categoría nueva no requiere migración.
- **Datos embebidos:** fotos, atributos y versiones de documentos viven dentro del mismo documento, así que la vista de detalle se arma con una sola consulta, sin joins.
- **Escrituras simples y abundantes:** notificaciones y registros de auditoría se generan todo el tiempo y no necesitan transacciones complejas.

- **Búsqueda:** permite índices sobre campos anidados (por ejemplo, `atributos.autor`) y búsqueda de texto.
- **Escala horizontal:** si el catálogo crece, se distribuye entre varios nodos (sharding).

Por qué cada base tiene un solo dueño

Es el patrón *database per service*. Si dos servicios compartieran base, quedarían acoplados: un cambio de esquema de uno rompería al otro, y ninguno podría desplegarse por separado. La relación entre ambos mundos es lógica: la ficha guarda un `subastaId`, pero el precio y las pujas nunca se copian a Mongo. El gateway junta ambos datos al responder.

Conectores

Usamos tres tipos distintos de conectores HTTP (GraphQL, WebSocket y REST), cada uno donde sus propiedades encajan. Un conector no es decorativo: define cómo se acoplan los componentes.

Conector	Entre	Estilo	Para qué
HTTP estático	Servidor web → navegador	Descarga de archivos	Entregar la SPA una vez
GraphQL	SPA → gateway	Consultas definidas por el cliente	Consultas y acciones de cada pantalla
WebSocket	SPA ↔ gateway	Canal bidireccional persistente	Precio en vivo, tiempo restante, "te superaron"
REST	Gateway → cada microservicio	Recursos + verbos HTTP	Operaciones de negocio
REST de eventos (outbox)	MS transaccional → gateway y MS catálogo	Entrega de eventos con reintento	Avisar lo que pasa sin petición de un usuario
Driver nativo	Cada servicio → su base	Protocolo propio de la base (no HTTP)	Leer y escribir datos

GraphQL (SPA → gateway)

Cómo funciona: el front envía por HTTP (un `POST` a un único endpoint) una consulta que describe exactamente los campos que quiere, y recibe un JSON con esa misma forma. Por ejemplo, la vista de detalle de una subasta:

```
query {
  subasta(id: 45) {
    precioActual
    cierraEn
    ultimasPujas(limite: 5) { alias monto }
    articulo { titulo fotos atributos }
  }
}
```

El gateway resuelve `precioActual` y `ultimasPujas` con el MS transaccional y `articulo` con el MS catálogo.

Por qué: cada pantalla necesita una combinación distinta de datos de ambos servicios. Con GraphQL el front los pide en una sola petición, sin traer de más ni de menos. Encaja con SOFEA: la presentación vive en el cliente, así que es el cliente quien sabe qué datos necesita cada vista. El esquema funciona además como contrato tipado entre front y back.

WebSocket (SPA ↔ gateway)

Cómo funciona: arranca como una petición HTTP con un *handshake* de "upgrade" y se convierte en un canal permanente y bidireccional. El servidor puede enviar datos al cliente sin que este los pida. Puede implementarse como GraphQL Subscriptions para quedar dentro del mismo esquema.

Por qué: el precio cambia por acciones de otros usuarios, y el negocio exige que quien es superado se entere inmediatamente. Sin WebSocket, el front tendría que preguntar cada pocos segundos si hay pujas nuevas (polling), lo cual es ineficiente y siempre llega tarde.

REST (gateway → microservicios)

Cómo funciona: HTTP con JSON, donde cada URL representa un recurso y el verbo indica la acción. Es síncrono, de petición-respuesta y sin estado. Ejemplos:

- MS transaccional: `POST /subastas`, `GET /subastas/{id}`, `POST /subastas/{id}/pujas`, `POST /billeteras/{id}/consignaciones`, `POST /ventas-directas/{id}/reservas`.
- MS catálogo: `GET /articulos?categoria=arte`, `GET /articulos/{id}`, `POST /piezas/{id}/documentos`, `GET /usuarios/{id}/notificaciones`.

Por qué REST aquí y no GraphQL: entre servicios no hace falta la flexibilidad de GraphQL. El gateway sabe exactamente qué necesita de cada servicio, así que una API de recursos estable es más simple de construir, probar (Postman, curl) y documentar (OpenAPI). El gateway habla el idioma de las pantallas; los servicios hablan el idioma del negocio y no saben nada de la interfaz.

REST de eventos (outbox)

Son las flechas punteadas del diagrama. El MS transaccional entrega por REST los eventos que genera por su cuenta: al MS catálogo (`POST /notificaciones`) para crear la notificación, y al gateway (`POST /internal/eventos`) para empujarla por WebSocket. El mecanismo completo se explica en la siguiente sección.

Drivers nativos

Cada servicio habla con su base por el protocolo propio de esta (el de PostgreSQL y el binario de MongoDB). No son conectores HTTP y no cuentan para ese requisito, pero conviene declararlos en la arquitectura.

Requisito de conectores

El enunciado pide al menos dos tipos distintos de conectores HTTP. Tenemos tres, y no son variantes de lo mismo: REST está orientado a recursos, GraphQL a consultas definidas por el cliente y WebSocket a comunicación bidireccional en tiempo real.

Cómo resolvemos lo que no puede fallar

Casi todos los riesgos que lista la descripción del negocio se resuelven con cinco mecanismos del MS transaccional: transacciones con bloqueo, actualizaciones condicionadas, restricciones en la base, un scheduler idempotente y el outbox.

Lo que no puede fallar	Mecanismo
Ofrecer más dinero del disponible	Validación de fondos dentro de la transacción de la puja
Descontar dos veces el dinero	Movimientos del ledger dentro de una sola transacción
Que el dinero no vuelva al ser superado	Liberación en la misma transacción que acepta la nueva puja
Dos ganadores del mismo lote	Bloqueo de la fila de la subasta y cierre condicionado

Lo que no puede fallar	Mecanismo
Vender una pieza dos veces	Actualización condicionada + índice único de venta activa
Dos compras directas reservando la misma pieza	Actualización condicionada del estado
Cerrar antes o después de la hora	Scheduler con la hora guardada en la base
Cerrar dos veces tras un reinicio	Cierre idempotente
Perder una puja válida al límite	Orden asignado por la base + extensión anti último segundo
Que una puja desaparezca o se modifique	Tablas de solo inserción
Perder un aviso tras una caída	Outbox

Pujas simultáneas

Si dos usuarios pujan al mismo tiempo, ambos podrían leer el mismo precio actual y creer que ganaron. Lo evitamos procesando cada puja dentro de una transacción que bloquea la fila de la subasta (`SELECT ... FOR UPDATE`): la segunda puja espera a que termine la primera y luego se valida contra el precio ya actualizado. Una alternativa es el control optimista con una columna de versión.

El orden lo asigna la base de datos (secuencia y hora del servidor dentro de la transacción), nunca el reloj del navegador. Así hay un criterio único para saber cuál llegó primero y se puede demostrar después.

Una pieza en una sola venta

Reservar o comprometer una pieza es una actualización condicionada:

```
UPDATE piezas SET estado = 'reservada'
WHERE id = :id AND estado = 'disponible';
```

Si dos compradores llegan a la vez, solo uno modifica la fila; el otro recibe cero filas afectadas y se le informa que la pieza ya no está disponible. Además, un índice único parcial impide que exista más de una venta activa por pieza, así que la regla se cumple aunque el código olvide revisarla.

Cierre automático e idempotente

La hora de cierre de cada subasta está guardada en la base. Un scheduler dentro del MS transaccional revisa periódicamente qué subastas vencieron y las cierra. El cierre no depende de que alguien tenga la página abierta.

El cierre también es condicionado:

```
UPDATE subastas SET estado = 'cerrada'
WHERE id = :id AND estado = 'abierta';
```

Si el servidor se reinicia, al volver el scheduler encuentra las subastas que vencieron mientras estaba caído y las cierra. Si una ya estaba cerrada, la condición impide cerrarla otra vez. La operación es **idempotente**: ejecutarla dos veces da el mismo resultado que ejecutarla una. El mismo mecanismo vence las reservas de venta directa y los plazos de pago.

Pujas de último segundo

Si una puja llega dentro de los últimos minutos, la misma transacción que la registra extiende la hora de cierre. Como la extensión y la puja ocurren juntas, el scheduler nunca puede cerrar la subasta "entre" una y otra.

Patrón outbox: que ningún evento se pierda

Hay cosas que ocurren sin que ningún usuario haga una petición: el cierre automático, el vencimiento de una reserva o de un plazo de pago. Alguien tiene que avisar al ganador y actualizar las pantallas en vivo.

1. En la misma transacción en que cierra la subasta (o acepta una puja), el MS transaccional inserta un evento en la tabla `eventos_salientes`, por ejemplo "subasta 45 cerrada, ganador X".
2. Un proceso del mismo servicio lee los eventos pendientes y los envía por REST al MS catálogo (notificación) y al gateway (WebSocket).
3. Solo marca el evento como enviado cuando recibe confirmación. Si algo falla, reintenta.

Por qué: el evento queda guardado en la misma transacción que el cambio que lo originó. Aunque el servidor se caiga justo después, el evento no se pierde. Como un reintento puede entregar el mismo evento dos veces, quien lo recibe lo identifica por su id y lo ignora si ya lo procesó.

La notificación nunca es la fuente de verdad: si un aviso no llega, el estado real está en la base y el usuario lo ve al entrar a la plataforma.

Ante la duda, detener

Si una operación encuentra un estado inconsistente, la transacción se aborta y la operación queda en `pendiente_revision` para un administrador. Es preferible una venta detenida que dos ganadores, dinero perdido o una pieza transferida a la persona equivocada.

Seguridad y privacidad

La seguridad se aplica en capas: el gateway filtra el acceso, los servicios validan la regla fina y los datos sensibles se cifran tanto al viajar como al almacenarse.

- **Cifrado en tránsito:** TLS entre el navegador y el sistema, terminado en el gateway. Dentro de la red de Docker, en el prototipo el tráfico entre servicios puede ir sin TLS; en producción se usaría mTLS.
- **Cifrado en reposo:** cifrado por campo para identidad, direcciones, datos financieros y de envío (`pgcrypto` en PostgreSQL y cifrado a nivel de campo en MongoDB). Los documentos sensibles se cifran antes de almacenarse.
- **Autenticación y roles:** JWT validado en el gateway. Cada rol tiene solo los permisos que necesita. Los servicios vuelven a validar las reglas de negocio aunque el gateway ya haya filtrado.
- **Información pública vs. privada:** la vista pública del catálogo no incluye datos del dueño. En el canal público de una subasta, los postores aparecen con alias.
- **Documentos protegidos:** nunca se exponen por URL directa. Se descargan a través del servicio, que valida permisos, o con URLs firmadas que vencen en minutos. Conocer la dirección no basta para descargarlos.
- **Historial inmutable:** pujas y movimientos de dinero son de solo inserción. Las correcciones son registros nuevos que dicen qué se corrigió, quién y por qué.
- **Auditoría administrativa:** retirar un lote, cambiar una configuración, anular una puja o desbloquear una cuenta queda registrado con autor, fecha y motivo.
- **Operaciones críticas:** cambiar el dueño de una pieza o retirar fondos requiere una sesión identificada y queda auditado. Una cuenta comprometida no basta para alterar el historial de una subasta, porque ese historial no se puede editar.
- **Rate limiting** en pujas e inicio de sesión contra manipulación y fuerza bruta.
- **Superficie expuesta mínima:** solo el servidor web y el gateway son accesibles desde afuera. Los microservicios y las bases solo existen en la red interna de los contenedores.

Despliegue en contenedores

Cada uno de los seis componentes corre en su propio contenedor Docker, y un `docker-compose.yml` los levanta juntos en una red interna con un solo comando.

Contenedor	Expuesto hacia afuera	Contenido
Servidor web	Sí	Archivos estáticos de la SPA
Gateway	Sí	Gateway de capa 7 / BFF
MS transaccional	No	Servicio + scheduler + publicador del outbox
MS catálogo y documentos	No	Servicio
PostgreSQL	No	Base del MS transaccional, con volumen persistente
MongoDB	No	Base del MS catálogo, con volumen persistente

Por qué:

- **Aislamiento:** cada componente tiene sus propias dependencias y versiones sin chocar con los demás. Esto también es lo que permite usar tres lenguajes distintos sin conflictos.
- **Reproducibilidad:** el sistema se levanta igual en el computador de cualquier integrante y en la máquina del profesor.
- **Distribución real:** cada contenedor es un proceso independiente que se comunica por red.
- **Escalado independiente:** si hay mucha carga de pujas, se levantan más réplicas solo de ese servicio. Con varias réplicas, el scheduler debe correr en una sola o tomar un bloqueo en la base para no cerrar la misma subasta dos veces; el cierre condicionado ya lo protege, pero conviene evitar trabajo repetido.
- **Seguridad:** solo dos contenedores reciben tráfico externo.
- **Persistencia:** las bases usan volúmenes, así que reiniciar un contenedor no borra datos ni el estado de las subastas.

Cumplimiento de requisitos

La arquitectura cumple los siete requisitos no funcionales del enunciado.

Requisito	Cómo se cumple
Arquitectura distribuida	Seis componentes independientes en contenedores separados, comunicados por red
≥ 1 componente de presentación (web)	Front-end SPA
≥ 2 componentes de lógica	Gateway, MS transaccional y MS catálogo y documentos
≥ 2 componentes de datos (relacional + NoSQL)	PostgreSQL y MongoDB
≥ 2 tipos de conectores HTTP	REST, GraphQL y WebSocket
≥ 3 lenguajes de propósito general	Propuesta: TypeScript (SPA y gateway), Go (MS transaccional) y Python (MS catálogo)
Despliegue orientado a contenedores	Docker + docker-compose

La elección de lenguajes es una propuesta. Go encaja en el MS transaccional por su manejo de concurrencia; Python encaja en el catálogo por la rapidez para construir APIs (FastAPI) y su buen soporte de MongoDB. Cualquier combinación de tres lenguajes distintos cumple el requisito.

Alcance propuesto del prototipo

Proponemos implementar completa la subasta sin evento y dejar el resto diseñado pero no implementado. La descripción del negocio corresponde a un sistema de producción, y el enunciado deja el alcance del prototipo a criterio del equipo.

Se implementa:

1. Registro de usuario y verificación simplificada.
2. Registro de pieza → revisión por un administrador → inventario disponible.
3. Publicación en el catálogo con atributos por categoría y fotos.
4. Billetera: consignar fondos y ver total, disponible y comprometido.
5. Subasta sin evento: pujar apartando dinero, liberar al ser superado, extensión anti último segundo y cierre automático.
6. Precio en vivo por WebSocket y notificaciones al superado y al ganador.

Por qué este recorte: ejercita todas las partes difíciles de la arquitectura (concurrencia, ledger, scheduler idempotente, outbox, los tres conectores y las dos bases) con el menor número de pantallas.

Queda diseñado, no implementado: subastas en evento en vivo, venta directa, asignación automática de peritos, proceso posterior a la venta, logística, reclamos y pago al vendedor.

Preguntas abiertas para el equipo

- ☐ ¿Estamos de acuerdo con el alcance propuesto, o incluimos venta directa?
- ☐ ¿Confirmamos los lenguajes (TypeScript, Go, Python) o cambiamos alguno?
- ☐ ¿Qué framework usamos para la SPA (React, Vue o Angular)?
- ☐ ¿Implementamos WebSocket como GraphQL Subscriptions o como WebSocket aparte?
- ☐ ¿Cómo simulamos la verificación de identidad en el prototipo?
- ☐ ¿Quién toma cada componente?